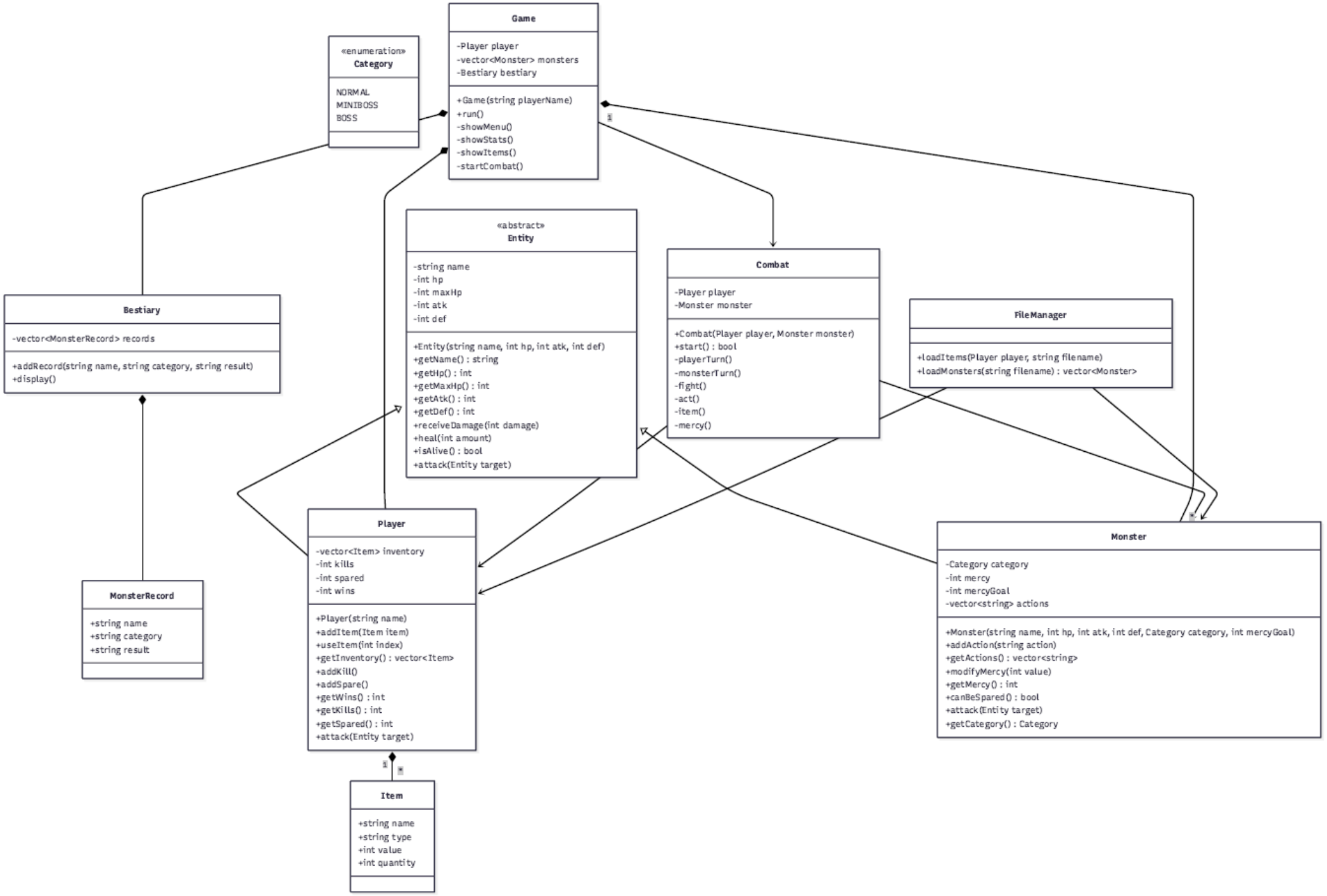
PROJET ALTERDUNE

GOUDON Octave – GOUEL Julien


Game
-Player player
-vector<Monster> monsters
-Bestiary bestiary
+Game(string playerName)
+run()
-showMenu()
-showStats()
-showItems()
-startCombat()
«enumeration»
Category
NORMAL
MINIBOSS
BOSS
«abstract»
Entity
-string name
-int hp
-int maxHp
-int atk
-int def
+Entity(string name, int hp, int atk, int def)
+getName() : string
+getHp() : int
+getMaxHp() : int
+getAtk() : int
+getDef() : int
+receiveDamage(int damage)
+heal(int amount)
+isAlive() : bool
+attack(Entity target)
Combat
-Player player
-Monster monster
+Combat(Player player, Monster monster)
+start() : bool
-playerTurn()
-monsterTurn()
-fight()
-act()
-item()
-mercy()
FileManager
+loadItems(Player player, string filename)
+loadMonsters(string filename) : vector<Monster>
Bestiary
-vector<MonsterRecord> records
+addRecord(string name, string category, string result)
+display()
MonsterRecord
+string name
+string category
+string result
Player
-vector<Item> inventory
-int kills
-int spared
-int wins
+Player(string name)
+addItem(Item item)
+useItem(int index)
+getInventory() : vector<Item>
+addKill()
+addSpare()
+getWins() : int
+getKills() : int
+getSpared() : int
+attack(Entity target)
Monster
-Category category
-int mercy
-int mercyGoal
-vector<string> actions
+Monster(string name, int hp, int atk, int def, Category category, int mercyGoal)
+addAction(string action)
+getActions() : vector<string>
+modifyMercy(int value)
+getMercy() : int
+canBeSpared() : bool
+attack(Entity target)
+getCategory() : Category
Item
+string name
+string type
+int value
+int quantity
1
*

Héritage via Entity

Nous avons utilisé la notation d'héritage via la classe Entity. En effet, les classes Player et Monster possèdent des attributs et des méthodes en commun et donc héritent de la classe Entity dans laquelle ces derniers sont définis.

```cpp
#define ENTITY_H

#include <string>

class Entity {
protected:
    std::string name;
    int hp;
    int maxHp;
    int atk;
    int def;
```

```cpp
#define MONSTER_H

#include "Entity.h"
#include <vector>
#include <string>

enum class Category { NORMAL, MINIBOSS, BOSS };

class Monster : public Entity {
private:
    Category category;
    int mercy;
    int mercyGoal;
    std::vector<std::string> actions;
```

```cpp
#define PLAYER_H

#include "Entity.h"
#include <vector>
#include <string>

struct Item {
    std::string name;
    std::string type;
    int value;
    int quantity;
};

class Player : public Entity {
private:
    std::vector<Item> inventory;
    int kills;
    int spared;
    int wins;
```

Encapsulation

Nous avons fait en sorte de mettre les attributs de chaque classe en protected. Concernant les méthodes/fonctions, cela dépend de l'usage, si l'utilisateur peut en faire appel ou non, nous avons mis public ou protected.

```
class Combat {
private:
    Player& player;
    Monster& monster;
    boo                                  // true si l
                class Monster
                Rechercher en ligne
    voi
    void monsterTurn();

    void fight();
    void act();
    void item();
    void mercy();

public:
    Combat(Player& p, Monster& m);

    bool start();        // retourne
    bool wasSpared() const;
};
```

Polymorphisme

Nous avons utilisé le polymorphisme via la méthode attack. En effet, lorsqu'un monstre attaque, cela ne fait pas les mêmes actions que pour le joueur.

```
virtual void attack(Entity& target) = 0;
```

```
void Player::attack(Entity& target) {
    int damage = rand() % (target.getMaxHp() + 1);

    std::cout << name << " attaque " << target.getName() << " !" << std::endl;

    if (damage == 0) {
        std::cout << "L'attaque echoue !" << std::endl;
    }
    else {
        target.receiveDamage(damage);
        std::cout << damage << " degats infliges !" << std::endl;
    }
}
```

```
void Monster::attack(Entity& target) {
    int damage = rand() % (target.getMaxHp() + 1);

    std::cout << name << " attaque !" << std::endl;

    if (damage == 0) {
        std::cout << "Le monstre rate son attaque !" << std::endl;
    }
    else {
        target.receiveDamage(damage);
        std::cout << damage << " degats subis !" << std::endl;
    }
}
```

Système ACT et Mercy

Nous avons créé un catalogue de 9 actions ACT assez originales qui ont un impact positif ou négatif sur Mercy.

```cpp
#include "Combat.h"
#include <iostream>
#include <map>
#include <random>

// Catalogue d'actions ACT (au moins 8, dont 2 impacts negatifs)
struct Act {
    std::string text;
    int mercyImpact;
};

std::map<std::string, Act> actCatalog = {
    {"JOKE",        {"Vous racontez une blague ridicule. Le monstre ricane malgre lui.",     +20}},
    {"COMPLIMENT",  {"Vous faites un compliment sincere. Le monstre rougit.",                +15}},
    {"INSULT",      {"Vous insultez violemment le monstre. Mauvaise idee.",                  -20}},
    {"DANCE",       {"Vous dansez bizarrement. Le monstre est perplexe... mais apaise.",     +10}},
    {"THREAT",      {"Vous menacez le monstre. Il ne semble pas apprecier.",                 -15}},
    {"PET",         {"Vous caressez le monstre. Il fait un bruit etrange et satisfait.",     +25}},
    {"OBSERVE",     {"Vous observez attentivement. Le monstre se sent compris.",              +5}},
    {"OFFER_SNACK", {"Vous offrez un snack. Le monstre le devore avec joie.",                +30}},
    {"REASON",      {"Vous tentez de raisonner le monstre. Il marque une pause...",          +20}}
};
```

Lecture des fichiers

Nous avons créé les fichiers monsters.csv et items.csv selon la consigne. Ainsi, nous les appelons à travers la classe FileManager afin d'y associer les éléments aux classes correspondantes.

```cpp
#ifndef FILEMANAGER_H
#define FILEMANAGER_H

#include "Player.h"
#include "Monster.h"
#include <vector>

class FileManager {
public:
    static void loadItems(Player& player, const std::string& filename);
    static std::vector<Monster> loadMonsters(const std::string& filename);
};

#endif
```

Implémentation du jeu

Menu Principal:

```
Entrez votre nom : joueur
L'humain joueur est tombe dans l'Alterdune.

=== BIENVENUE DANS ALTERDUNE ===
Joueur : joueur
HP : 100/100
Inventaire de depart :
  - Potion x3 (Soin : +15 HP)
  - Snack x5 (Soin : +8 HP)
  - SuperPotion x1 (Soin : +30 HP)
  - MegaPotion x1 (Soin : +20 HP)
================================

--- MENU PRINCIPAL --- (Victoires : 0/10)
1. Bestiaire
2. Demarrer un combat
3. Statistiques
4. Items
5. Quitter
Choix :
```

Système de Combat:

```
Choix : 2

--- COMBAT CONTRE Froggit ---

Player HP: 100/100 | Monster HP: 30/30 | Mercy: 0

FIGHT | ACT | ITEM | MERCY
FIGHT
joueur attaque Froggit !
27 degats infliges !

Tour du monstre...
Froggit attaque !
86 degats subis ! HP restants : 14

Player HP: 14/100 | Monster HP: 3/30 | Mercy: 0

FIGHT | ACT | ITEM | MERCY
ACT
Actions disponibles :
0 : COMPLIMENT
1 : JOKE
1
Vous racontez une blague ridicule. Le monstre ricane malgre lui.
Mercy : 20

Tour du monstre...
Froggit attaque !
27 degats subis ! HP restants : 0
Vous etes mort... GAME OVER
```